Case Study 1: RISC vs. CISC — Choosing a Processor

Computer Organization and Architecture (25CST-238) | Shriya, Roll No. 25BAI10016 | B.Tech CSE (AI), Chandigarh University

1. Scenario and Problem Statement

A technology company has built two computing systems, one on a RISC architecture and the other on a CISC architecture. The team has to choose a suitable instruction-set design for a **low-power embedded device**. The problem is to compare the two processor architectures and justify the preferred option for this use case. This report answers the five case study questions, then gives a comparison table, two example instruction sequences and a reasoned recommendation.

2. Answers to the Case Study Questions

Q1. What is RISC? What is CISC?

RISC (Reduced Instruction Set Computer) uses a small set of simple, uniform instructions, most of which complete in one clock cycle. Only LOAD and STORE instructions access memory; all other work happens between registers (examples: ARM, RISC-V, MIPS). **CISC (Complex Instruction Set Computer)** provides a large set of instructions, some of which perform several steps at once, such as reading memory, calculating and writing back in one instruction. These take a variable number of cycles (examples: Intel x86, VAX, Motorola 68000). In short, RISC keeps the hardware simple and leaves the work to the compiler, while CISC puts more work into the hardware.

Q2. Why do many RISC instruction sets use fixed-length instructions?

When every instruction has the same size (32 bits in MIPS, RISC-V and classic ARM), the processor knows where the next instruction starts without examining the current one. Fetching can run ahead, the decoder stays simple, and the pipeline does not stall working out instruction boundaries. Instructions also align neatly in memory and the cache. The cost is slightly larger programs, which is why some designs add short 16-bit forms (ARM Thumb-2, RISC-V compressed) while keeping decoding easy.

Q3. How does instruction complexity influence instruction decoding?

The more varied the instructions, the harder decoding becomes. An x86 instruction can be 1 to 15 bytes long with prefixes, optional fields and many operand forms, so the decoder must first find the length before it can locate the next instruction. This needs more logic, more transistors, more power, and often microcode for the complex cases. RISC decoding mostly means reading fixed bit fields (opcode, registers, immediate), which is fast and cheap. Modern x86 chips hide this cost by translating instructions into simple internal micro-operations, but the translator itself consumes area and energy.

Q4. Compare the typical addressing-mode approaches in RISC and CISC.

RISC offers few modes: register, immediate and base-plus-offset, and only load/store instructions use memory addressing, so arithmetic is register-to-register. CISC offers many modes (direct, indirect, indexed, base + index x scale + displacement, even memory-to-memory) and most instructions may use them. CISC is therefore more flexible and gives shorter code, whereas RISC is easier to implement, pipeline and time. (Some RISC chips such as ARM add a few extras like auto-increment, but still far fewer than x86.)

Q5. Can a CISC processor outperform a RISC processor? Explain with one reason.

Yes, because of code density. One CISC instruction can replace several RISC instructions, so the program is smaller and needs less instruction-fetch bandwidth. When memory is slow or the instruction cache is small, fewer bytes fetched can make the CISC machine faster. Modern x86 chips also combine this with RISC-like internal execution.

3. (a) Comparison Table

Parameter	RISC	CISC
Instruction set size	Small and simple	Large, includes complex instructions
Instruction length	Mostly fixed (e.g. 32-bit)	Variable (1-15 bytes in x86)
Cycles per instruction	Usually 1 (pipeline-friendly)	Varies, often several
Memory access	Only via LOAD / STORE	Most instructions can use memory operands
Addressing modes	Few: register, immediate, base+offset	Many and complex
Register file	Large (e.g. 32 GPRs)	Smaller (e.g. 8-16 GPRs)
Decoder / control unit	Simple, usually hardwired	Complex, often microprogrammed
Code size	Larger	Smaller (higher code density)
Pipelining	Easy and efficient	Harder because of uneven instructions
Power and chip area	Lower	Higher
Compiler role	Compiler does more optimisation work	Hardware does more per instruction
Typical examples	ARM, RISC-V, MIPS	Intel x86, VAX, Motorola 68000

3. (b) Example Instruction Sequences

Example 1: counter = counter + 1 (a variable stored in memory)

CISC (x86), 1 instruction:

```
ADD  DWORD PTR [counter], 1        ; read memory, add 1, write back
```

RISC (ARM), 3 instructions:

```
LDR  R1, [R0]        ; load counter from memory (R0 holds its address)
ADD  R1, R1, #1      ; add 1 in a register
STR  R1, [R0]        ; store the result back
```

The CISC version is shorter, but it hides three steps inside one instruction with a variable cycle count. The RISC version needs more instructions, but each is fixed in size, takes about one cycle and moves smoothly through the pipeline.

Example 2: copying a block of N bytes

CISC (x86), 1 instruction (after setting ESI = source, EDI = destination, ECX = N):

```
REP MOVSB            ; repeat: copy a byte, advance pointers, ECX = ECX - 1
```

RISC (ARM), a short loop (R0 = source, R1 = destination, R3 = N):

```
loop: LDRB R2, [R0], #1   ; load a byte, then move source pointer
      STRB R2, [R1], #1   ; store the byte, then move destination pointer
      SUBS R3, R3, #1     ; decrement count and set flags
      BNE  loop           ; repeat until count is zero
```

Again CISC wins on code size (one instruction against four), while RISC gives simple, predictable steps that are easy to pipeline and time exactly.

3. (c) Recommendation for a Battery-Powered Embedded Device

Recommended choice: a RISC processor, such as an ARM Cortex-M or RISC-V microcontroller. The reasons are:

1. **Lower power consumption.** A simple decoder and fixed-length instructions mean fewer transistors switching per instruction, so less energy is drawn from the battery.
2. **Smaller, cheaper chip.** Less control logic gives a smaller die, lower cost and less heat, so no fan or heat sink is needed in a compact device.
3. **Predictable timing.** Embedded tasks (sensors, motor control) often have real-time deadlines. Mostly single-cycle instructions make execution time easy to calculate.
4. **Efficient pipelining at low clock speed.** Uniform instructions keep the pipeline busy, so useful work is done without a high clock rate, and a lower clock saves further power.
5. **Mature ecosystem.** ARM and RISC-V have broad compiler, library and microcontroller support, with sleep modes designed for battery life.

Trade-off and mitigation: RISC programs are larger and need more instruction fetches. Flash memory is cheap, and compressed instruction sets (ARM Thumb-2, RISC-V "C" extension) recover most of the gap. CISC advantages such as raw speed on complex workloads matter little for a device that mainly needs long battery life.

Conclusion: CISC tries to do more per instruction, while RISC tries to do each instruction cheaply and quickly. For a low-power, battery-powered embedded device, the simplicity, efficiency and predictability of RISC make it the better choice.